Self-Healing Data Pipeline

Demo Walkthrough — Cortex AI-Powered Schema Drift Detection & Automated Remediation

Christian Abdel-Nour, Sr. Technical Architect — Snowflake Solutions Delivery (Public Sector)

July 2026 • Duration: ~15 minutes • Repo: github.com/sfc-gh-cabdelnour/selfhealing-pipeline

The Problem

When upstream data sources change their schema (columns added, dropped, or type-changed), downstream pipelines break silently. Engineers discover the issue hours or days later via failed dashboards, incorrect aggregates, or user complaints. The investigation and fix cycle is manual, repetitive, and error-prone.

This pipeline automates the entire remediation lifecycle:

- Detects schema drift within 15 minutes (not hours/days)
- Halts the pipeline to prevent broken data from propagating
- Diagnoses the root cause using Cortex AI
- Regenerates impacted downstream SQL using an LLM
- Validates the fix against a zero-copy clone of production
- Opens a GitHub PR for human approval

> **Design Rationale: Why Build This Inside Snowflake?**
>
> Every component (Tasks, stored procedures, Cortex AI, CLONE, External Access Integration) is a first-party Snowflake feature. This means: no external orchestrator (Airflow, Lambda), no credential rotation for CI tools, no network egress except the GitHub API call, and the same RBAC governs the pipeline as governs the data. The pattern is portable to any Snowflake account.

Demo Flow

1. Show the Healthy Baseline

Open Snowsight and connect to the sandbox. Set context to `SELFHEALING_PROD.CONFIG`. Show the audience the current state.

Talk Track

"Here's our production pipeline — 3 BRONZE tables (raw source data), SILVER views (cleansed), and 2 GOLD aggregate tables. Everything is stable and healthy."

```
-- Show the pipeline objects
SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_TYPE
FROM SELFHEALING_PROD.INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA IN ('BRONZE','SILVER','GOLD')
ORDER BY TABLE_SCHEMA, TABLE_NAME;

-- Show the schema registry baseline (what the system "expects")
SELECT TABLE_NAME, COLUMN_NAME, DATA_TYPE
FROM SELFHEALING_PROD.CONFIG.SCHEMA_REGISTRY
WHERE TABLE_NAME = 'ORDERS'
ORDER BY ORDINAL_POSITION;

-- Show the lineage graph (what depends on what)
SELECT * FROM SELFHEALING_PROD.CONFIG.VW_ARTIFACT_LINEAGE;
```

> **Design Rationale: Why a Schema Registry (Not Just INFORMATION_SCHEMA)?**
>
> INFORMATION_SCHEMA shows what IS. The registry stores what SHOULD BE — the last known-good state. The diff between the two is drift. Without a baseline, you can't distinguish "column was added today" from "column has always been there." The registry advances only after a human merges the fix, creating a deliberate ratchet.

2. Simulate an Upstream Schema Change

Talk Track

"A source system just added a new column. Maybe a vendor added a discount code to their API, or Finance added a field to their Excel workbook."

```
ALTER TABLE SELFHEALING_PROD.BRONZE.ORDERS ADD COLUMN DISCOUNT_CODE VARCHAR;
```

> **Design Rationale: Why ALTER TABLE (Not Openflow CDC)?**
>
> In production, schema changes arrive automatically via Openflow connectors. For the demo, we simulate with DDL. The pipeline is source-agnostic — it only cares about the diff between INFORMATION_SCHEMA and the registry. The detection logic is identical regardless of how the column appeared.

3. Run the Drift Detector

Talk Track

"The detector runs every 15 minutes on a scheduled task. Let's trigger it manually and see what it finds."

```
USE SCHEMA SELFHEALING_PROD.CONFIG;
CALL SP_SCHEMA_DRIFT_DETECTOR();

-- See the detected event
SELECT EVENT_ID, CHANGE_TYPE, TABLE_NAME, COLUMN_NAME, PIPELINE_STATUS
FROM SCHEMA_CHANGE_EVENTS;
```

Expected: `NEW_COLUMN | ORDERS | DISCOUNT_CODE | PENDING`

> **Design Rationale: Why Idempotent Detection?**
>
> The detector inserts only events that don't already have a PENDING/CODE_GENERATED/PR_OPEN entry for the same table+column+type. Running it 10 times with no new changes produces 0 new events. This is critical for scheduled tasks — if the task fires twice (transient failure + retry), it must not create duplicates.

4. Demonstrate the Halt Gate

Talk Track

"Before any pipeline refresh can proceed, we check the gate. While there are unresolved drift events, the pipeline is blocked."

```
CALL SP_HALT_GATE();
```

Expected: `HALTED — 0 critical, 1 pending`

> **Design Rationale: Why a Halt Gate (Not Just Alerting)?**
>
> Alerting tells a human something is wrong. Halting prevents broken data from propagating while the human is asleep/busy/on-PTO. This was an explicit requirement from the customer architect: "detect schema drift in real time, halt ingestion, present recommended actions, require manual approval before merging changes."

5. Generate Code with Cortex AI

Talk Track

"Now we ask Cortex AI to regenerate the impacted models. First it walks the lineage graph to find what's downstream, then it regenerates each model's SQL using an LLM."

```
-- Get the event ID
SET eid = (SELECT EVENT_ID FROM SCHEMA_CHANGE_EVENTS WHERE PIPELINE_STATUS='PENDING' LIMIT
1);

-- Show lineage: what's impacted?
SELECT * FROM VW_ARTIFACT_LINEAGE WHERE CHANGED_TABLE = 'ORDERS';

-- Generate (calls Cortex llama3.1-70b for SQL regeneration)
CALL SP_GENERATE_ARTIFACT_CODE($eid);

-- Show the generated SQL
SELECT ARTIFACT_NAME, ARTIFACT_SCHEMA, LEFT(GENERATED_SQL, 200) AS SQL_PREVIEW
FROM GENERATED_CODE WHERE EVENT_ID = $eid;
```

Design Rationale: Why Two Different LLMs?

claude-4-sonnet for diagnosis (needs strong reasoning to explain root cause in structured JSON). **llama3.1-70b** for code generation (needs speed and low cost for mechanical SQL rewrites). This matches Snowflake's Cortex best practices: "Start with the most capable models to establish a baseline, then optimize for cost." Diagnosis runs once per event; code gen runs once per impacted model.

Design Rationale: Why a Recursive CTE for Lineage (Not dbt Manifest)?

A recursive CTE over ARTIFACT_REGISTRY is pure SQL — no external manifest parsing, no Python dependency, no dbt CLI required. The view is always current (any change to the registry is instantly reflected). It finds ALL downstream models, not just direct dependents, so a BRONZE change correctly propagates through SILVER to GOLD. The pattern works whether you use dbt, stored procedures, or Dynamic Tables.

6. Validate Against a Zero-Copy Clone

Talk Track

"Before we trust the generated code, we test it. We create a zero-copy clone of production (instant, zero storage cost), execute the generated SQL against it, and record pass/fail per model."

```
CALL SP_VALIDATE_GENERATED_CODE($eid);

-- Check results
SELECT ARTIFACT_NAME, TEST_STATUS, LEFT(TEST_ERROR, 100) AS ERROR
FROM GENERATED_CODE WHERE EVENT_ID = $eid;

-- Final pipeline status
SELECT PIPELINE_STATUS FROM SCHEMA_CHANGE_EVENTS WHERE EVENT_ID = $eid;
```

Expected: `ORDERS_DAILY | FAIL` and status = `CI_FAILED`

> **Design Rationale: Why Zero-Copy Clone (Not a Staging Environment)?**
>
> Snowflake's CLONE creates an instant, full-fidelity copy of production at zero storage cost (copy-on-write). This means we test generated code against real data structures without maintaining a separate staging environment, without data sync jobs, and without touching production. The clone is dropped immediately after validation to prevent storage accumulation. Total cost: ~0.001 credits per validation.

7. Explain the CI_FAILED Result

Talk Track

"The GOLD model failed because it's an aggregate query (GROUP BY ORDER_DATE). The LLM added DISCOUNT_CODE to the SELECT but didn't add it to the GROUP BY — because whether to group by it, aggregate it, or exclude it is a *business decision*. This is exactly why we have the human gate."

> **Key Demo Insight**
> If everything passed, the audience would ask "why do we need humans?" The failure demonstrates that AI handles mechanical changes (pass-through columns in SILVER) perfectly, but cannot infer business intent for aggregations. The PR-first pattern ensures a human makes that decision — not the machine.

8. Architecture and Design Decisions

Decision	Choice	Rationale
Language for detection	SQL	Pure set operations on INFORMATION_SCHEMA — no iteration, fastest cold-start
Language for codegen	Python (Snowpark)	Reliable iteration, proper error handling, cleaner Cortex integration
LLM for diagnosis	claude-4-sonnet	Best reasoning for structured root cause JSON
LLM for code gen	llama3.1-70b	Good SQL generation, lower cost, sufficient for mechanical rewrites
Lineage storage	Table + recursive view	SQL-native, no manifest parsing, always current
Validation	Zero-copy clone + execute	Real data, zero storage cost, instant, no staging env
Human gate	PR-first (GitHub EAI)	Auditable, reviewable, reversible via standard git
Scheduling	Snowflake Task (15 min)	Native, no external scheduler, auto-suspend warehouse
Idempotency	Skip-if-exists on all writes	Safe for re-execution after transient failures
Error handling	TRY_PARSE_JSON + try/except	Graceful degradation — logs errors, never crashes on LLM issues

Q&A Preparation

Likely Question	Answer
"What if the LLM generates wrong SQL that still compiles?"	Validation catches syntax errors. Semantic correctness is the human's job at PR review. That's why we never auto-merge.
"Can this auto-merge if tests pass?"	Architecturally yes. We deliberately chose not to because even passing code can have wrong business logic.
"How does this handle multiple simultaneous changes?"	Events processed one at a time (LIMIT 1). Sequential to avoid stale-branch conflicts.

"Does this work with dbt?"	Yes — generated SQL is committed to the dbt project and validated via EXECUTE DBT PROJECT. Demo uses direct SQL for simplicity.
"Can this handle data quality issues, not just schema drift?"	Yes — PIPELINE_HEALTH_AUDIT also captures null FK rates, duplicate keys, orphan FKs. Same halt gate pattern.

Closing Statement

"This is a spike — a working proof of concept deployed on our sandbox in under 3 hours using Cortex Code. The pattern is Snowflake-native, uses existing Cortex AI capabilities, and integrates with GitHub for change management. If the team wants this on the customer environment, it's an afternoon of adaptation work."